

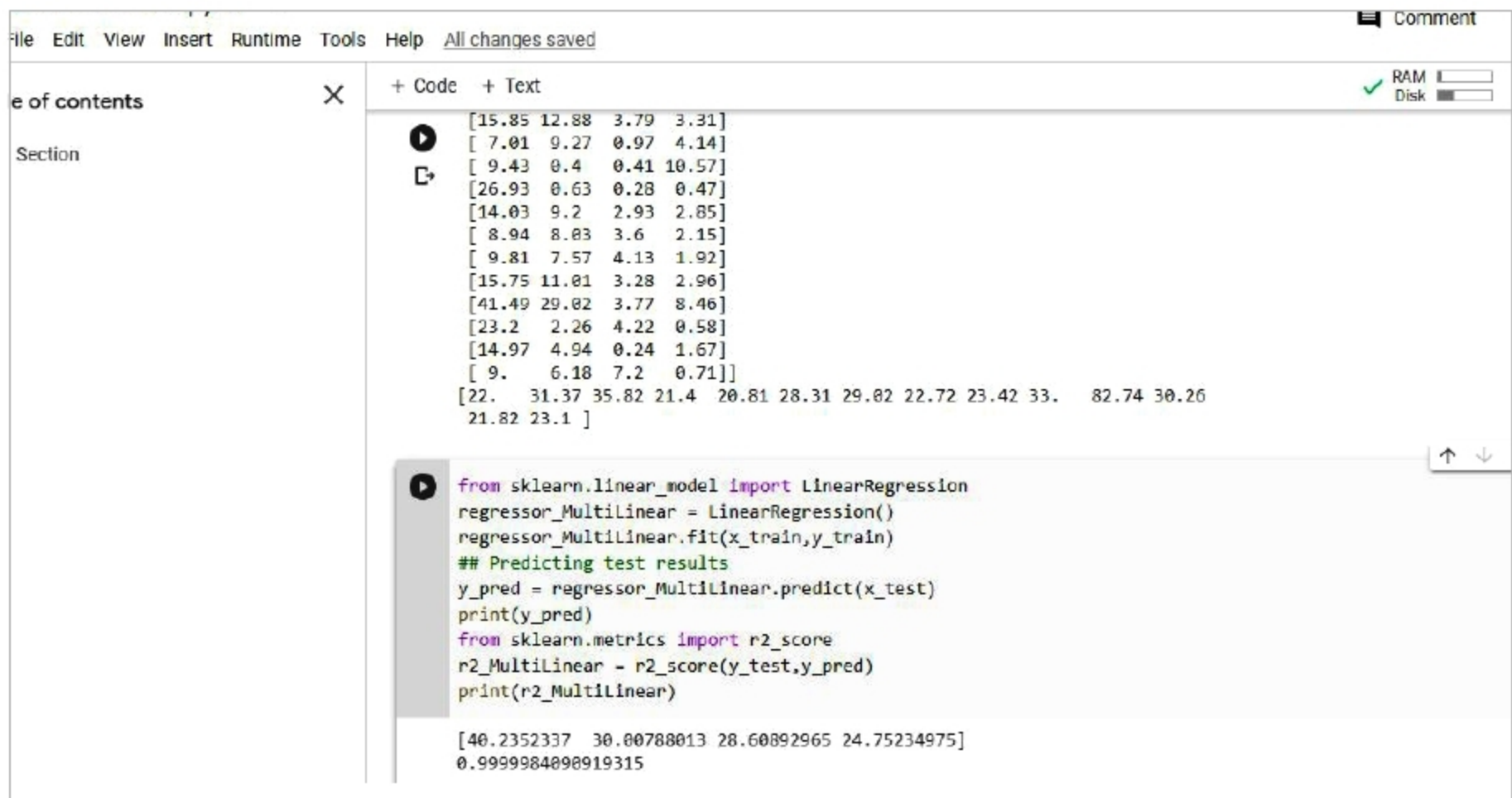


Figure 6: Colab screenshot of the predicted sales volume and degree of coefficient

evaluate the result to calculate the outcome of the variables. The statistical method heat map is a great technique used to visualise the correlation matrix of data for feature selection to solve business problems. We can see from the heat map in Figure 4 that dependent features like *NA_Sales*, *JP_Sales*, *EU_Sales* and *Other_Sales* are more contextual and relevant for the analysis.

Now, as the correlated feature set, which impacts the main analysis, is identified, we will focus on the actual algorithm and model selection, which may run in multiple iterations. Generally, we split the actual data set into a train, test it in an 80:20 ratio and iteratively run the different regression techniques. The *train:test* split can be easily executed with the Python library *sklearn*, as shown below:

```

#from sklearn.model_selection import
train_test_split
x_train,x_test,y_train,y_test = train_
test_split(x,y,test_size=0.2,random_
state=0)

```

Figure 5 shows the Google Colab screenshot, where we have uploaded the data set csv file from the local drive and executed it. We can see on the top right corner the allocated disk and RAM for executing the Python codebase below, while the highlighted section provides a method to choose the file through a browser pop-up to upload the data file *vg-sales.csv* from Kaggle.

We are now in the ultimate phase of executing the algorithm, which is readily available from the Python *sklearn* library to train the model with the training data set for multiple linear regression. You can see the standard package for it, imported from the *sklearn* library:

```

from sklearn.linear_model import
LinearRegression
regressor_Multilinear =
LinearRegression()
regressor_Multilinear.fit(x_train,y_
train)

```

Once the model is trained, we will predict the actual test results and *r2* coefficient, which is an effective

way to find and compare the accuracy while applying various techniques of regression. The best possible score is 1.0, and it can be negative if the model selected is arbitrarily worse. The standard definition of the *r2* score in statistics is the proportion of the variance in the dependent variable that is predictable from the independent variable(s).

```

## Predicting test results
y_pred = regressor_Multilinear.
predict(x_test)
# Calculating r2 score
from sklearn.metrics import r2_score
r2_Multilinear = r2_score(y_test,y_
pred)
print(r2_Multilinear)

```

There are several standard models associated with the regression technique like polynomial regression, XGBoost, decision tree, K-Nearest neighbour regression, random forest regression, linear support vector regression, non-linear support vector, etc.

Continued on page 62....